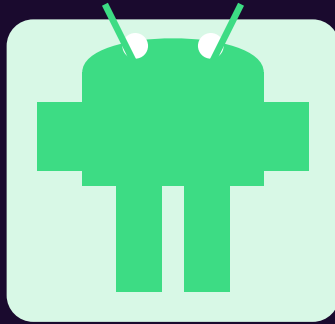




Clash Royale Julian

Documentación Técnica — Parcial 2

Autor: Julian Kristof Márquez Morales

Profesor: Adolfo Centeno

Entrega: 8 de Mayo de 2026

Flutter | Spring Boot | PostgreSQL | Docker | Render

0 Contents

1	Descripcion General del Proyecto	3
2	Tecnologias Utilizadas	3
3	Patrones de Diseño Utilizados	3
4	Patron Estructural: Facade	4
4.1	Definicion del patron Facade	4
4.2	Estructura general del patrón	4
4.3	Por que se uso en este proyecto?	4
4.4	Problema sin Facade vs. solución con Facade	5
4.5	Codigo implementado: TweetService	5
4.6	Codigo implementado: Controladores que usan la Facade	7
4.7	Diagrama UML del patrón Facade en el proyecto	9
4.8	Beneficios obtenidos del patrón Facade	10
5	Patron Repository	10
5.0.1	Repositorios implementados	10
6	Modelos / Entidades JPA	11
7	Patron MVC	13
8	Patron DTO	13
8.1	Aplicación en el proyecto	13
9	Patron Dependency Injection	14
9.1	Aplicación en el proyecto	14
9.2	Beneficio	15
10	Patron Singleton	15
10.1	Aplicación en el proyecto	15
10.2	Beneficio	15
11	Patron Strategy	15
11.1	Aplicación en el proyecto	15
12	Patron Builder	16
12.1	Aplicación en el proyecto	16
12.2	Beneficio	16
13	Patron Proxy	16
13.1	Aplicación en el proyecto	16
14	Endpoints de la API REST	17
15	Arquitectura y Despliegue	17

16 Conclusion

18

1 Descripción General del Proyecto

Resumen: Aplicación multiplataforma para registrar cartas y contenido relacionado con Clash Royale, con soporte de reacciones emoji y comentarios por publicación. Backend REST con Spring Boot y frontend Flutter compilado para web, Linux desktop y APK Android.

La idea del proyecto es tener una especie de red social enfocada en cartas y contenido de Clash Royale. Cada usuario puede crear una publicación con el nombre de la carta, su costo de elixir, rareza, imagen y demás datos. Otros usuarios pueden reaccionar con emojis (*like*, *love*, *haha*, *wow*, *sad*) y también dejar comentarios de texto.

El backend se hizo con **Spring Boot** y expone una API REST. La base de datos es **PostgreSQL** en producción (Aiven). El frontend se construyó con **Flutter**, compilando la misma base de código para web y APK de Android. El proyecto está desplegado en **Render** usando **Docker**.

2 Tecnologías Utilizadas

Tecnología	Uso en el proyecto
Flutter (Dart)	Interfaz: web, Linux desktop y APK Android
Spring Boot 4	Framework backend, expone la API REST
PostgreSQL 16	Base de datos relacional en producción y local
Hibernate / JPA	Maapeo objeto-relacional de las entidades
Docker	Contenedorización del backend para despliegue
Render	Plataforma de despliegue en la nube
Maven	Gestión de dependencias y build del proyecto Java
GitHub	Control de versiones

Table 1: Stack tecnológico del proyecto

3 Patrones de Diseño Utilizados

En el proyecto se utilizaron nueve patrones de diseño y arquitectura: **Facade**, **Repository**, **MVC**, **Dependency Injection**, **DTO**, **Singleton**, **Strategy**, **Builder** y **Proxy**. La combinación de estos patrones permitió mantener el código ordenado, desacoplado y más fácil de escalar.

4 Patron Estructural: Facade

¿Qué es un patrón estructural?

Los **patrones estructurales** se ocupan de cómo se componen las clases y los objetos para formar estructuras más grandes. No se enfocan en crear objetos (eso es trabajo de los patrones creacionales) ni en cómo interactúan en tiempo de ejecución (eso es trabajo de los comportamentales). Su objetivo es simplificar las relaciones entre clases, reducir el acoplamiento y hacer que el sistema sea más fácil de entender y mantener. Ejemplos de patrones estructurales del catálogo GoF: **Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy**.

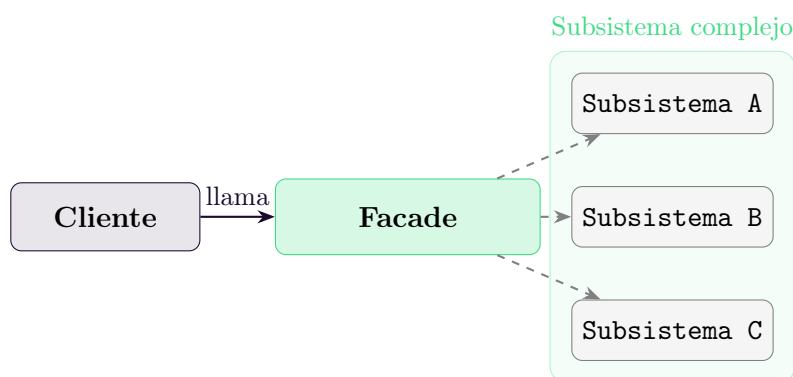
4.1 Definición del patron Facade

¿Qué es el patrón Facade?

Facade (Fachada) proporciona una interfaz simplificada a un conjunto de interfaces en un subsistema. Define una interfaz de nivel superior que hace que el subsistema sea más fácil de usar. En lugar de que el cliente tenga que conocer y coordinar múltiples clases internas, habla con una sola clase fachada que lo resuelve todo internamente.

La analogía clásica es la de un cajero automático: el usuario solo presiona botones, sin saber que por detrás hay comunicación con el banco, verificación de saldo, actualización de registros contables y emisión de comprobantes. Toda esa complejidad está *detrás de la fachada*.

4.2 Estructura general del patrón



4.3 Por que se uso en este proyecto?

Sin un Facade, cada controlador REST tendría que importar directamente `TweetRepository`, `TweetCommentRepository` y `TweetReactionRepository`, y además conocer la lógica de cómo buscar una carta, validar que existe, crear la entidad relacionada y guardarla. Eso genera **acoplamiento fuerte**: si cambia la lógica de almacenamiento, hay que modificar cada controlador.

Con `TweetService`, los controladores solo conocen a la fachada y le delegan todo. La fachada *esconde* la complejidad de coordinar varios repositorios y concentrar la lógica de negocio.

4.4 Problema sin Facade vs. solución con Facade

Sin Facade: controlador acoplado a múltiples repositorios

```
1 // SIN FACADE - el controlador tiene que conocer TODOS los
  repositorios
2 @RestController
3 public class ComentarioController {
4     // Acoplado a 3 repositorios directamente
5     private final TweetRepository tweetRepo;
6     private final TweetCommentRepository comentarioRepo;
7     private final TweetReactionRepository reaccionRepo;
8
9     @PostMapping("/{id}/comentarios")
10    public ResponseEntity<?> agregar(@PathVariable Long id,
11                                    @RequestBody Map<String,
12    String> body) {
13        // Logica duplicada en cada controlador
14        Tweet tweet = tweetRepo.findById(id)
15            .orElseThrow(() -> new RuntimeException("No existe
16    "));
17        TweetComment c = new TweetComment(body.get("texto"),
18    tweet);
19        comentarioRepo.save(c);
20        return ResponseEntity.ok(c);
21    }
22 }
```

Con Facade: controlador simple y lógica centralizada

```
1 // CON FACADE - el controlador solo conoce a TweetService
2 @RestController
3 public class ComentarioController {
4     private final TweetService tweetService; // UNA sola
5     dependencia
6
7     @PostMapping("/{id}/comentarios")
8     public ResponseEntity<?> agregar(@PathVariable Long id,
9                                     @RequestBody Map<String,
10    String> body) {
11        // Solo delega, no sabe como funciona internamente
12        TweetComment c = tweetService.addComment(id, body.get(
13    "texto"));
14        return ResponseEntity.ok(c);
15    }
16 }
```

4.5 Código implementado: `TweetService`

Esta es la clase central del dominio. Coordina los repositorios internamente y expone métodos simples y cohesivos:

TweetService.java -- La Fachada

```
1 @Service
2 public class TweetService {
3
4     private final TweetRepository tweetRepository;
5     private final TweetCommentRepository tweetCommentRepository;
6     private final TweetReactionRepository tweetReactionRepository
7     ;
8     private final TweetMapperStrategy tweetMapper;
9
10    public TweetService(TweetRepository tweetRepository,
11                        TweetCommentRepository
12                        tweetCommentRepository,
13                        TweetReactionRepository
14                        tweetReactionRepository,
15                        TweetMapperStrategy tweetMapper) {
16        this.tweetRepository = tweetRepository;
17        this.tweetCommentRepository = tweetCommentRepository;
18        this.tweetReactionRepository = tweetReactionRepository;
19        this.tweetMapper = tweetMapper;
20    }
21
22    public Page<TweetResponseDTO> getTweets(Pageable pageable) {
23        return tweetRepository.findAll(pageable)
24            .map(tweetMapper::toResponseDTO);
25    }
26
27    public TweetResponseDTO createTweet(Tweet tweet, String
28    username) {
29        tweet.setUsername(username);
30        Tweet saved = tweetRepository.save(tweet);
31        return tweetMapper.toResponseDTO(saved);
32    }
33
34    public TweetResponseDTO reactToTweet(Long tweetId, String
35    username,
36
37    TweetReactionRequest
38    request) {
39        Tweet tweet = tweetRepository.findById(tweetId)
40            .orElseThrow(() -> new RuntimeException("Tweet no
41            encontrado"));
42        TweetReaction reaction = new TweetReaction(request.
43            getReactionType(),
44
45            username,
46            tweet);
47        tweetReactionRepository.save(reaction);
48        return tweetMapper.toResponseDTO(tweet);
49    }
50
51    public TweetCommentResponseDTO addComment(Long tweetId,
52    String username,
53
54    TweetCommentRequest
55    request) {
56        Tweet tweet = tweetRepository.findById(tweetId)
57            .orElseThrow(() -> new RuntimeException("Tweet no
58            encontrado"));
59    }
```

```

44         TweetComment comment = new TweetComment(request.
getContent(),
45                                                     username, tweet);
46         tweetCommentRepository.save(comment);
47         return new TweetCommentResponseDTO(
48             comment.getId(),
49             comment.getContent(),
50             comment.getUsername()
51         );
52     }
53
54     public List<TweetCommentResponseDTO> getComments(Long tweetId
) {
55         return tweetCommentRepository.findByTweetId(tweetId).
stream()
56             .map(comment -> new TweetCommentResponseDTO(
57                 comment.getId(),
58                 comment.getContent(),
59                 comment.getUsername()))
60             .toList();
61     }
62 }

```

4.6 Código implementado: Controladores que usan la Facade

TweetController.java

```

1  @CrossOrigin(origins = "*")
2  @RestController
3  @RequestMapping("/api/tweets")
4  public class TweetController {
5
6      private final TweetService tweetService;
7
8      public TweetController(TweetService tweetService) {
9          this.tweetService = tweetService;
10     }
11
12     @GetMapping("")
13     public Page<TweetResponseDTO> getTweet(Pageable pageable) {
14         return tweetService.getTweets(pageable);
15     }
16
17     @PostMapping("")
18     public ResponseEntity<TweetResponseDTO> createTweet(
19         @Valid @RequestBody Tweet tweet,
20         Authentication authentication) {
21         return ResponseEntity.status(201)
22             .body(tweetService.createTweet(tweet,
authentication.getName()));
23     }
24
25     @DeleteMapping("/{id}")
26     public ResponseEntity<Void> deleteTweet(
27         @PathVariable Long id,

```



```
28         Authentication authentication) {
29         tweetService.deleteTweet(id, authentication.getName());
30         return ResponseEntity.noContent().build();
31     }
32 }
```

TweetCommentController.java

```
1  @CrossOrigin(origins = "*")
2  @RestController
3  @RequestMapping("/api/tweets")
4  public class TweetCommentController {
5
6      private final TweetService tweetService;
7
8      public TweetCommentController(TweetService tweetService) {
9          this.tweetService = tweetService;
10     }
11
12     @GetMapping("/{tweetId}/comments")
13     public ResponseEntity<List<TweetCommentResponseDTO>>
14     getComments(
15         @PathVariable Long tweetId) {
16         return ResponseEntity.ok(tweetService.getComments(tweetId));
17     }
18
19     @PostMapping("/{tweetId}/comments")
20     public ResponseEntity<TweetCommentResponseDTO> addComment(
21         @PathVariable Long tweetId,
22         @Valid @RequestBody TweetCommentRequest request,
23         Authentication authentication) {
24         return ResponseEntity.status(201)
25             .body(tweetService.addComment(
26                 tweetId,
27                 authentication.getName(),
28                 request));
29     }
30 }
```

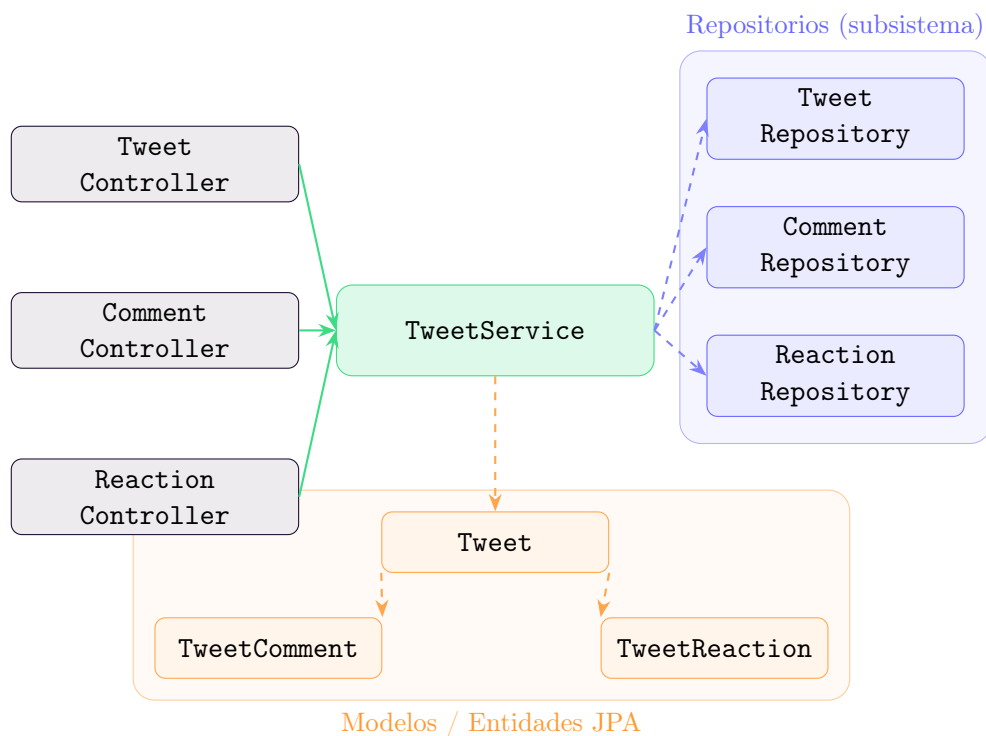
TweetReactionController.java

```
1  @CrossOrigin(origins = "*")
2  @RestController
3  @RequestMapping("/api/tweets")
4  public class TweetReactionController {
5
6      private final TweetService tweetService;
7
8      public TweetReactionController(TweetService tweetService) {
9          this.tweetService = tweetService;
10     }
11
12     @PostMapping("/{tweetId}/reactions")
13     public ResponseEntity<TweetResponseDTO> reactToTweet(
```

```
14         @PathVariable Long tweetId,  
15         @Valid @RequestBody TweetReactionRequest request,  
16         Authentication authentication) {  
17  
18         return ResponseEntity.ok(  
19             tweetService.reactToTweet(  
20                 tweetId,  
21                 authentication.getName(),  
22                 request)  
23             );  
24     }  
25 }
```

4.7 Diagrama UML del patrón Facade en el proyecto

El siguiente diagrama muestra cómo TweetService actúa como la única puerta de entrada que los controladores usan para acceder al subsistema de repositorios y modelos:



4.8 Beneficios obtenidos del patrón Facade

Beneficio	Impacto en el proyecto
Bajo acoplamiento	Los controladores no dependen de los repositorios directamente
Único punto de cambio	Si cambia la lógica de comentarios o reacciones, solo se modifica <code>TweetService</code>
Código limpio	Cada controlador queda pequeño y directo
Lógica centralizada	Las reglas del dominio viven en un solo lugar
Fácil de testear	Se puede hacer mock de <code>TweetService</code> sin levantar repositorios

Table 2: Beneficios del patrón Facade en el proyecto

5 Patron Repository

¿Qué es el patrón Repository?

El Repository actúa como una colección de objetos en memoria. En lugar de escribir SQL directamente en la lógica de negocio, se crea una interfaz que define qué operaciones se pueden hacer con cada entidad. Spring Data JPA genera automáticamente la implementación a partir del nombre de los métodos, sin necesidad de escribir SQL.

Repositorios implementados

TweetRepository.java

```

1 @Repository
2 public interface TweetRepository
3     extends JpaRepository<Tweet, Long> {
4
5     Page<Tweet> findAll(Pageable pageable);
6 }
```

TweetCommentRepository.java

```

1 @Repository
2 public interface TweetCommentRepository
3     extends JpaRepository<TweetComment, Long> {
4
5     List<TweetComment> findByTweetId(Long tweetId);
6 }
```

TweetReactionRepository.java

```
1 @Repository
2 public interface TweetReactionRepository
3     extends JpaRepository<TweetReaction, Long> {
4
5     List<TweetReaction> findByTweetId(Long tweetId);
6     long countByTweetId(Long tweetId);
7 }
```

UserRepository.java

```
1 @Repository
2 public interface UserRepository
3     extends JpaRepository<User, Long> {
4
5     Optional<User> findByUsername(String username);
6
7     boolean existsByUsername(String username);
8
9     boolean existsByEmail(String email);
10 }
```

6 Modelos / Entidades JPA

Los modelos son clases Java anotadas con `@Entity` que Hibernate mapea automáticamente a tablas de PostgreSQL.

Tweet.java -- Entidad principal

```
1 @Entity
2 @Table(name = "tweets")
3 public class Tweet {
4
5     @Id
6     @GeneratedValue(strategy = GenerationType.IDENTITY)
7     private Long id;
8
9     private String nombre;
10
11     private String rareza;
12
13     private Integer costoElixir;
14
15     @Column(length = 1000)
16     private String urlImagen;
17
18     private String username;
19 }
```

TweetComment.java

```
1 @Entity
2 @Table(name = "tweet_comments")
3 public class TweetComment {
4
5     @Id
6     @GeneratedValue(strategy = GenerationType.IDENTITY)
7     private Long id;
8
9     private String content;
10
11     private String username;
12
13     @ManyToOne(fetch = FetchType.LAZY)
14     @JoinColumn(name = "tweet_id")
15     @JsonIgnore
16     private Tweet tweet;
17 }
```

TweetReaction.java

```
1 @Entity
2 @Table(name = "tweet_reactions")
3 public class TweetReaction {
4
5     @Id
6     @GeneratedValue(strategy = GenerationType.IDENTITY)
7     private Long id;
8
9     private String reactionType;
10
11     private String username;
12
13     @ManyToOne(fetch = FetchType.LAZY)
14     @JoinColumn(name = "tweet_id")
15     @JsonIgnore
16     private Tweet tweet;
17 }
```

User.java

```
1 @Entity
2 @Table(name = "users")
3 public class User {
4
5     @Id
6     @GeneratedValue(strategy = GenerationType.IDENTITY)
7     private Long id;
8
9     @NotBlank
10    private String username;
11
12    @NotBlank
13    @Column(unique = true)
14    private String email;
15 }
```

```
16     @NotBlank
17     private String password;
18 }
```

7 Patron MVC

¿Qué es el patrón MVC?

MVC divide la aplicación en tres capas: el **Modelo** maneja los datos, la **Vista** muestra la interfaz al usuario, y el **Controlador** recibe las peticiones y coordina entre los otros dos. Esto permite cambiar la interfaz sin tocar la lógica del negocio.

En este proyecto MVC se aplica en dos niveles simultáneos:

- **Backend (Spring Boot):** las entidades JPA son el Modelo, los `@RestController` son los Controladores, y el JSON que retorna la API es la representación de la Vista.
- **Frontend (Flutter):** los widgets de Flutter son la Vista que consume la API y muestra los datos al usuario.

8 Patron DTO

¿Qué es un DTO?

DTO significa *Data Transfer Object*. Se utiliza para transferir únicamente la información necesaria entre backend y frontend sin exponer directamente las entidades internas de la base de datos.

8.1 Aplicación en el proyecto

El sistema utiliza DTOs para respuestas y peticiones, lo que permite controlar exactamente qué campos viajan por la API.

TweetResponseDTO.java

```
1 public class TweetResponseDTO {
2
3     private Long id;
4     private String nombre;
5     private Integer costoElixir;
6     private String rareza;
7     private String urlImagen;
8     private String username;
9 }
```

TweetCommentResponseDTO.java

```
1 public class TweetCommentResponseDTO {
2
3     private Long id;
4     private String content;
5     private String username;
6 }
```

TweetReactionRequest.java

```
1 public class TweetReactionRequest {
2
3     private String reactionType;
4 }
```

9 Patron Dependency Injection

¿Qué es Dependency Injection?

Dependency Injection permite que Spring inyecte dependencias automáticamente mediante el constructor. Esto reduce el acoplamiento entre clases y facilita el mantenimiento y las pruebas.

9.1 Aplicación en el proyecto

Todos los controladores y servicios utilizan inyección de dependencias.

AuthController.java

```
1 @RestController
2 @RequestMapping("/api/auth")
3 public class AuthController {
4
5     private final AuthService authService;
6
7     public AuthController(AuthService authService) {
8         this.authService = authService;
9     }
10 }
```

TweetController.java

```
1 private final TweetService tweetService;
2
3 public TweetController(TweetService tweetService) {
4     this.tweetService = tweetService;
5 }
```

9.2 Beneficio

La lógica queda desacoplada y se pueden cambiar implementaciones sin modificar el resto de las clases.

10 Patron Singleton

¿Qué es Singleton?

Singleton garantiza que una clase tenga una única instancia compartida durante toda la ejecución de la aplicación.

10.1 Aplicación en el proyecto

Spring maneja los beans como Singleton por defecto. En particular, los servicios, controladores y repositorios se crean una sola vez y se reutilizan durante toda la ejecución.

TweetService.java

```
1 @Service
2 public class TweetService {
3 }
```

10.2 Beneficio

Evita crear objetos innecesarios y mantiene una instancia compartida de los componentes principales.

11 Patron Strategy

¿Qué es Strategy?

Strategy permite intercambiar algoritmos o reglas de comportamiento sin modificar al cliente que los usa.

11.1 Aplicación en el proyecto

Spring Security utiliza distintas estrategias internas para autenticación y autorización mediante JWT y roles.

TestController.java

```
1 @GetMapping("/admin")
2 @PreAuthorize("hasRole('ADMIN')")
3 public String adminAccess() {
4     return "Clash Royale admin board.";
5 }
```



```
5 }
```

Cada ruta aplica una estrategia diferente de autorización según el rol del usuario.

12 Patron Builder

¿Qué es Builder?

Builder facilita la construcción de objetos complejos paso a paso y mejora la legibilidad del código.

12.1 Aplicación en el proyecto

Spring usa un estilo fluido en la construcción de respuestas HTTP con `ResponseEntity`.

ResponseEntity

```
1 return ResponseEntity.status(201)
2     .body(tweetService.createTweet(tweet, authentication.
    getName()));
```

```
1 return ResponseEntity.noContent().build();
```

12.2 Beneficio

El código queda más expresivo y se entiende con claridad el tipo de respuesta que se está construyendo.

13 Patron Proxy

¿Qué es Proxy?

Proxy controla el acceso a otro objeto y puede aplicar validaciones o filtros antes de ejecutar la lógica real.

13.1 Aplicación en el proyecto

Spring Security actúa como proxy antes de permitir acceso a endpoints protegidos, interceptando peticiones y verificando el token JWT y los roles.

TestController.java

```
1 @PreAuthorize("hasRole('ADMIN')")
2 public String adminAccess() {
3     return "Clash Royale admin board.";
4 }
```

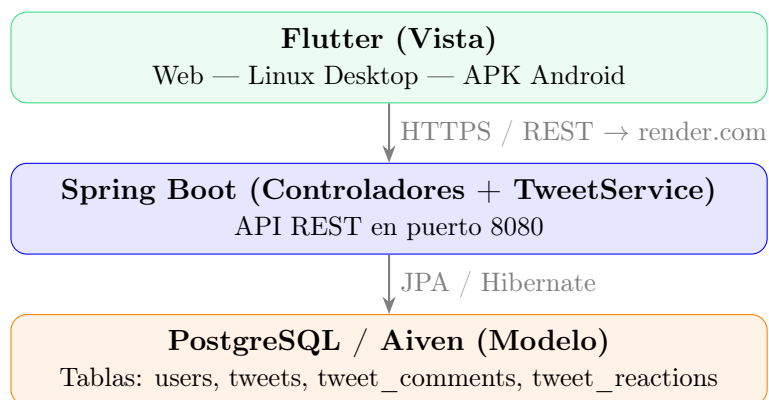
Spring intercepta la petición antes de ejecutar el método real.

14 Endpoints de la API REST

Metodo	Ruta	Descripcion
Autenticación		
POST	/api/auth/signin	Iniciar sesión
POST	/api/auth/signup	Registrar usuario
Tweets / Cartas		
GET	/api/tweets	Listar publicaciones paginadas
POST	/api/tweets	Crear nueva publicación
DELETE	/api/tweets/{id}	Eliminar publicación
Reacciones y comentarios		
POST	/api/tweets/{id}/reactions	Agregar reacción
GET	/api/tweets/{id}/comments	Lista de comentarios
POST	/api/tweets/{id}/comments	Agregar comentario

Table 3: Endpoints principales de la API REST

15 Arquitectura y Despliegue



El backend se empaqueta con Maven en un JAR y se contenedorizó con Docker. Las variables de entorno `DATABASE_URL`, `DB_USER` y `DB_PASSWORD` configuran la conexión a Aiven desde Render.

```
# Desarrollo local
$ mvn spring-boot:run -Dspring-boot.run.profiles=local

# Producción (Render ejecuta el backend con Docker)
$ flutter build web -release # frontend web
$ flutter build apk -release # APK Android
```

16 Conclusion

El patrón **Facade** fue el aporte estructural más importante del proyecto. Al encapsular toda la coordinación entre `TweetRepository`, `TweetCommentRepository` y `TweetReactionRepository` dentro de `TweetService`, se logró que los controladores REST quedaran con una sola dependencia y con una estructura mucho más limpia. Eso hace el sistema más fácil de mantener: si la lógica de comentarios o reacciones cambia, solo se modifica la fachada.

Los **repositorios** eliminaron la necesidad de escribir SQL, ya que Spring Data JPA genera las consultas automáticamente a partir del nombre de los métodos como `findByTweetId` o `existsByUsername`.

El patrón **MVC** permitió desarrollar el backend (Spring Boot) y el frontend (Flutter) de forma completamente independiente, comunicándose únicamente a través de la API REST desplegada en Render.